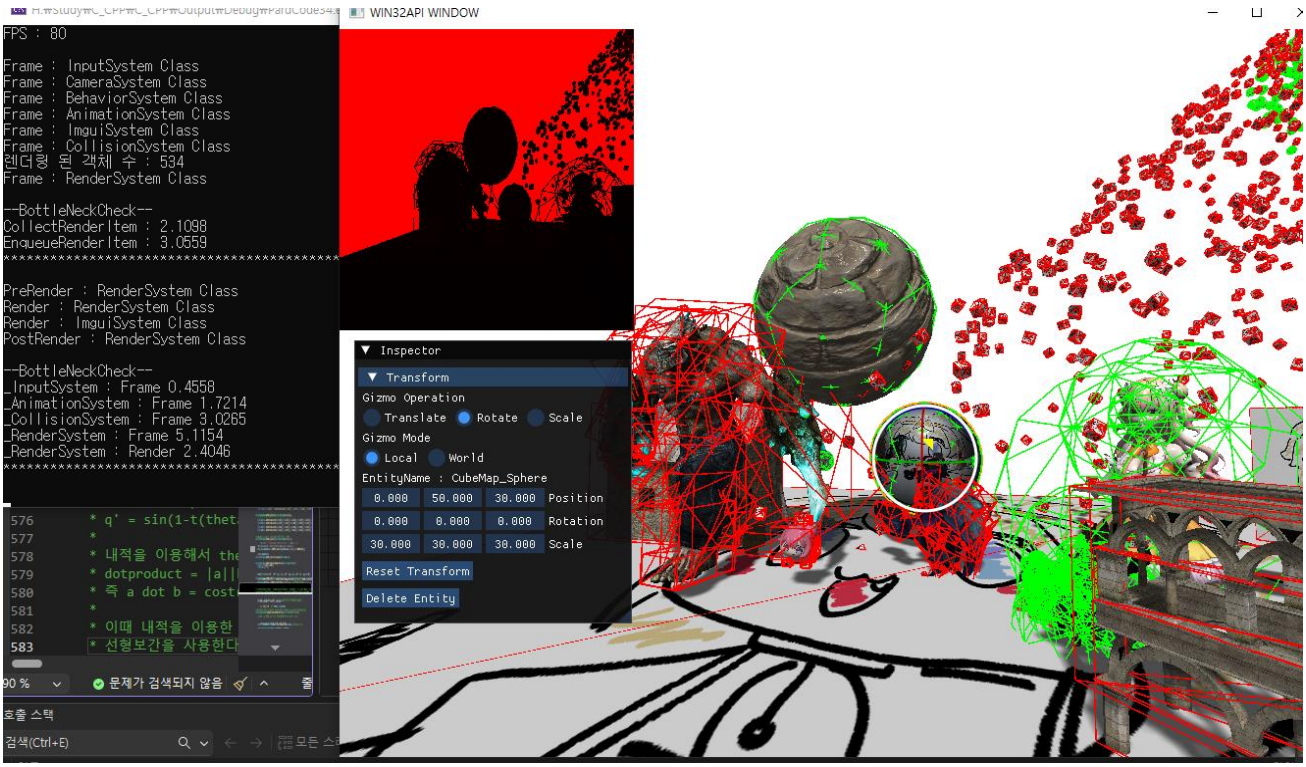
PERSONAL PROJECT PORTFOLIO

[DX11기반 ECS아키텍쳐 렌더러]

[데이터 지향 설계(DOD)와 렌더 패스 파이프라인으로 구성한 캐시 최적화 렌더링 프로젝트]



PERIOD	약 1년 (2025.05 ~), 개인학습 프로젝트, 단계적 구조 개선 진행
ROLE	1인개발 – 아키텍쳐 설계, 엔진/렌더링 시스템구현, 디버깅 및 최적화
STACK	C++17, DX11/HLSL, Assimp, ImGui, CRTP, ECS
SUMMARY	단계별로 직접 마주한 구조적문제(자원관리, 비효율적 파이프라인 바인딩, OOP기반 렌더링의 캐시 비친화성 등) 을 근거로 엔진 설계를 진행한 과정을 기록한 작업일지를 기반으로 핵심 기술을 위주로 재구성한 문서
GitHub	https://github.com/ckdlscjs/c_cpp

00 [프로젝트 개요]

문제인식 및 개발배경

본 프로젝트는 DirectX 11과 C++17을 기반으로 처음부터 직접 구현한 3D 렌더링 엔진으로 단순히 "동작하는 렌더러"를 만드는 것을 넘어, **왜 이 구조를 선택했는지에 대한 문제의식과 설계 근거**를 기록하는 것을 목표로 진행하였다

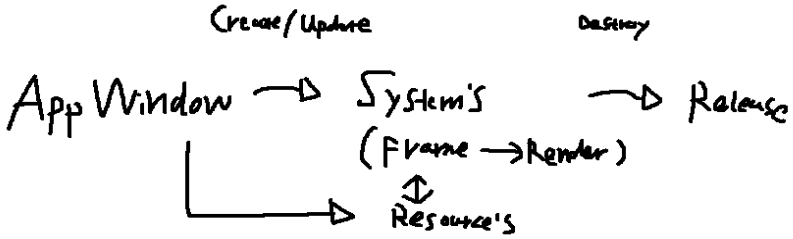
과거 진행했던 팀프로젝트 단위의 렌더러/툴/게임 작업에서 엔진의 일부분이나 단편적인 기능만을 구현했고 그래픽스 엔진의 세부구성이나 파이프라인 전반을 관통하는 설계를 작업하지 못해 기술적 갈증과 아쉬움이 남았고 이를 처음부터 구현함으로써 현대적인 그래픽스 기술을 사용하는 엔진을 목표로 기술을 학습하기위해 렌더러 프로젝트를 시작하였다

프로젝트 목표

과거 객체지향(OOP) 디자인 패턴을 활용해 개발했던 이전 DirectX 프로젝트에서의 경험이 본 엔진 아키텍처의 근본적인 출발점이 되었다. 당시 수천 개의 동적 오브젝트를 배치했을 때 발생했던 메모리 파편화와 드로우콜 병목 한계를 직접 체감하였고, 리소스 자원 관리 및 캐시 친화적 설계를 하기위해 **CRTP 기반 시스템 설계 → DOD 기반 ECS 아키텍처 전환 → 렌더패스/정렬키 기반 렌더링 파이프라인 구축**까지 단계적으로 구조를 발전시키는 방식으로 점진적인 기술 습득 및 이를 적용하는 방식으로 현재의 결과물을 완성했다

전체 아키텍처 흐름

본 프로그램은App Window → EngineSystem(자원 로드/생성) → ECS System(엔티티/컴포넌트 생성)의 초기화 흐름을 거치며, 매 프레임 각 시스템의 **Frame** → **Render** 단계를 통해 상태 갱신(Update)과 렌더 명령 제출(Render)단계로 수행된다. 프로그램 종료 시점에는 RAII에 기반해 획득한 객체에서 자원을 소멸시킨다



핵심 시스템 구성

기반구조	CRTP 기반 BaseSystem / BaseView 템플릿 - 모든 매니저 시스템과 D3D 리소스 래퍼의 공통 기반 수립
수학기반	DirectXMath 기반 Vector2~4 / Matrix4x4 / Quaternion 자체 래퍼 구현 (16바이트 정렬 및 SIMD 가속)
ECS(DOD)	Archetype + Bitset 기반 데이터지향(DOD) 엔티티-컴포넌트 시스템, 16KB Chunk 단위 메모리 직렬화 관리
렌더링 기반	RenderItem 수집 → 64비트 정렬 키 → 렌더패스(Deferred/Shadow/Outline 등) 단위 드로우콜 일괄 처리
충돌/컬링	Frustum Culling, Ray-OBB 1차 피킹 및 Compute Shader(Möller-Trumbore) 기반 정밀 폴리곤 교차 판정
리소스	D3D11래퍼자원 및 가상자원 리소스 구성(Geometry/Mesh/Texture/Material/Animation) 파이프라인 구축 및 FNV-1a 해시 기반 중복 로드 제거
기타 시스템	Input(Polling/Event), Behavior, Timer, ImGui 디버그 오버레이 연동

01 [엔진 아키텍쳐 및 기반시스템]

[1.CRTP 기반 시스템 설계]

문제인식 및 필요성

- Engine, Resource, ECS, Render 등 고유성이 보장되어야 하는 매니저 클래스들의 싱글톤 보일러플레이트 코드 중복 제거 필요.
- D3D11 객체들의 명확한 수명 관리 부재 시 발생하는 자원 누수를 일률적인 패턴으로 통제 필요.

설계

- **CRTP(Curiously Recurring Template Pattern) 적용**: 공통 기능을 제공하는 기본 시스템 클래스 템플릿 설계.
- **지연 생성(Lazy Initialization)**: static T instance 방식을 채택하여 인스턴스 최초 사용 시점에 생성 유도 (C++11 표준 기반의 스레드 안전성 보장).
- **RAII 패턴 준수**: 리소스 클래스들의 생성/소멸 시점을 동기화하여 수명 관리를 직관적으로 제어.

[2.수학 라이브러리 정의(Vector, Matrix, Quaternion)]

문제인식 및 필요성

- DirectXMath의 XMVECTOR, XMMATRIX는 SIMD 레지스터 가속 이점을 가지나, 연산자 오버로딩 제약과 디버깅 시 멤버 변수 접근의 불편함 잔재.
- 3D 좌표계 변환 및 셰이더 연산용 자체 수학 인프라를 구축하여 개발 효율성과 가독성 확보 필요.

설계

- **자체 수학 래퍼 구현**: Vector2~4, Matrix4x4, Quaternion 구현 및 연산자 오버로딩 구성.
- **16바이트 정렬**: __declspec(align(16)) 지시어를 수립하여 SIMD 레지스터 적재 효율 보존.
- **공용체(Union) 레이아웃**: 내부적으로 XMMATRIX/XMVECTOR를 유지하되, 개별 컴포넌트 첨자(_11 ~ _44) 접근을 허용하여 디버깅 및 수정 용이성 확보.
- **사원수(Quaternion) 도입**: 오일러각의 한계인 짐벌락(Gimbal Lock)을 방지하고 구면선형보간(Slerp) 연산 수립.
- **전역 연산 인프라**: 카메라 시스템, 월드 변환(Transform), 절두체 컬링, 경계 볼륨 충돌 및 키프레임 애니메이션 보간의 핵심 자료구조로 활용.

[3. 가상자원 및 D3D Wrapper정의]

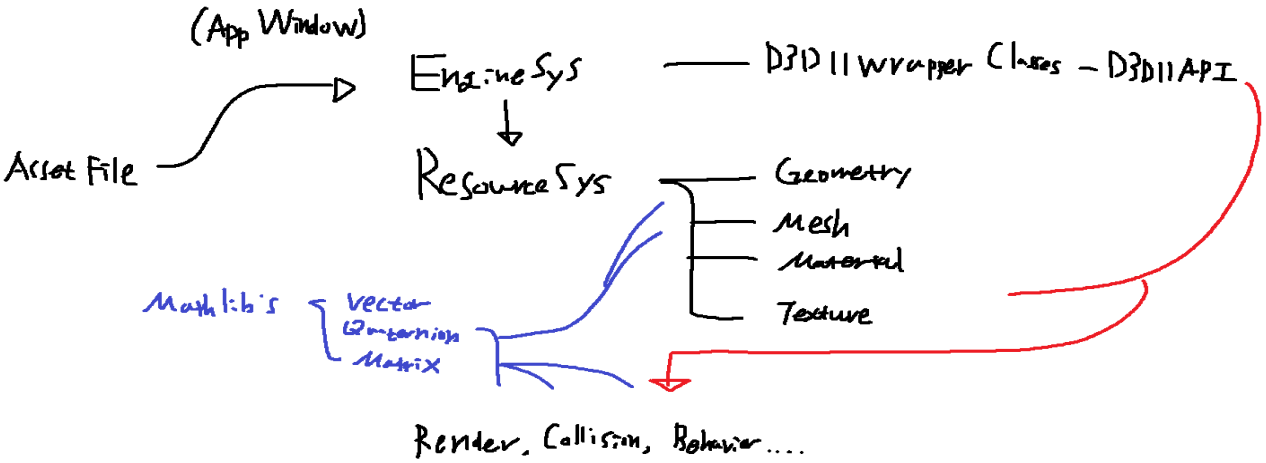
문제인식 및 필요성

- 텍스처, 기하(FBX/OBJ), 머티리얼, 애니메이션 등 고수준 데이터를 가공하여 D3D11 버퍼 및 뷰 객체로 변환하는 과정에서 중복 로드 및 불필요한 재생성 방지 필요.
- 초기 설계의 객체별 독립적 자원(버퍼/뷰/상태) 소유 구조는 중복 바인딩 및 컨텍스트 상태 전환 병목을 유발하므로, 자원 관리 주체와 사용 주체의 명확한 분리 필요.

설계

- **역할 분리**: 가상 정의 자원(Texture, Geometry, Mesh, Material, Asset)을 설계하고, D3D11 물리 자원은 EngineSystem, 고수준 에셋 관리는 ResourceSystem으로 이원화.
- **FNV-1a 해싱 기반 캐싱**: 파일 경로/에셋명을 FNV-1a 64비트 해시 키로 변환하여 중복 로드를 O(1)에 차단하고 유일한 객체 포인터 반환.
- **Texture**: DirectXTex 기반 로드 후, D3D11 리소스 뷰 변환 해시를 소유하여 파이프라인에서 공유.
- **Geometry**: Assimp를 연동해 노드 계층 구조를 재귀 순회(ParseNode)하고, 커스텀 해시 함수로 중복 정점을 제거하여 기하 데이터(Vertex/Index/Bone/Animation) 구성.
- **Mesh**: 가공된 Geometry 기반으로 D3D11 버퍼(VB/IB)를 생성 및 캐싱하여 바인딩 효율 극대화.

- **Material**: 파이프라인 바인딩용 셰이더 및 렌더 상태(States) 정보의 결합 구조 정의.
- **D3D11 Wrapper Classes**: BaseBuffer<T>, BaseState<T>, BaseView<T> 템플릿을 기반으로 GPU 자원 업로드 및 렌더 상태 캐싱 컨테이너 구현 (EngineSystem에서 리포지터리 패턴으로 일원화 관리).



[4. SubSystem's (Input, Behavior, Timer, Imgui)]

문제인식 및 필요성

- 입력 처리, UI 도구, 시간 측정 등 핵심 렌더링 리소스와 무관하지만 프레임 제어 및 에디터 구성에 필수적인 보조 유틸리티 시스템들의 유기적인 통합과 결합도 최소화 필요.

설계

BaseSystem<T> 기반으로 독립적인 4대 엔진 유틸리티 구축.

- **InputSystem**: Win32 메시지 루프를 바이패스하여 가상 키(VK) 상태를 실시간 폴링하는 구조와 특정 키 트리거 시 콜백을 호출하는 이벤트 구동식(Event-Driven) 리스너 구조 동시 지원.
- **BehaviorSystem**: ECS 청크 데이터를 순회하며 움직임 컴포넌트가 활성화된 개체의 위치 갱신 및 빌보드 행렬 연산 처리.
- **TimerSystem**: Win32 API 고정밀 성능 카운터 기반의 DeltaTime 측정, FPS 계산 및 ScopedTimer를 이용한 개별 함수 소요 시간 프로파일링 지원.
- **ImguiSystem**: ImGui 및 ImGuizmo를 연동하여 기즈모 조작, 선택 개체 Transform 실시간 제어 등 씬 에디터 조작 인프라 구축.

결과 및 개선여지

종합 활용 결과

- 싱글톤 템플릿 및 RAII 자원 관리 체계 일원화로 보일러플레이트 제거 및 자원 누수 가능성 차단.
- 독립적 책임 중심의 서브시스템 설계로 코드 간 결합도(Coupling)를 낮추고 편집 편의성 확보.
- 해시 기반 자원 매니저를 통해 중복 I/O 및 데이터 중복 업로드를 방지하고 중복 정점을 사전에 필터링하여 파이프라인 부하 경감.
- 고유 수학 라이브러리를 통해 가속 연산 성능 확보 및 직관적인 수학 디버깅 환경 조성.

향후 개선 여지

- **에셋 파이프라인 포맷 최적화**: Assimp 런타임 파싱 비용을 줄이기 위해 엔진 최적화 포맷으로 사전 직렬화하고 비동기 병렬 적재 파이프라인 구축 필요.
- **수학 연산 고도화**: 복사 비용 절감을 위한 더티 플래그 캐싱 도입 및 SIMD 명령어 극대화 최적화 여지 존재.

개발과정 세부사항

수학 클래스 구성 및 정의

사원수 클래스 구성 및 정의

Assimp라이브러리 임포트 및 애니메이션(Geometry 등 가상자원)

챕터1 오버뷰 및 세부 코드 스니펫

02 [DOD기반 EntityComponentSystem]

[1. OOP기반의 한계성과 DOD기반으로의 전환]

문제인식 및 필요성

- **메모리 파편화 및 캐시 미스**: OOP 기반의 개별 객체(TempObj)들이 힙(Heap) 주소 공간에 산발적으로 파편화 배치되어 CPU 캐시 미스를 유발하고 성능 저하 초래.
- **파이프라인 바인딩 오버헤드**: 파편화된 데이터 조합에 의해 렌더링 파이프라인에서 동일 자원의 바인딩 교체가 빈번히 발생하여 드로우콜 성능 저하 유발.

설계

- **DOD(DataOrientedDesign) ECS 리팩토링**: 파이프라인 흐름과 연속된 데이터 집합에 초점을 맞춘 데이터 지향 구조 구축.
- **Entity**: 데이터나 로직을 소유하지 않고, 고유 식별 정보(ID)만 저장하는 초경량 주소 역할 정의.
- **Component**: 자신이 소유하는 데이터만 지닌 순수 데이터 구조체. 캐시 히트율 극대화를 위해 메모리상에 조밀 선형 배열(DenseArray) 형태로 정렬 배치.
- **System**: 필요한 컴포넌트 조합(Dense Array 묶음)을 쿼리하여 순회 연산을 실행하는 실질적 기능 연산 주체.
- **Archetype / ArchetypeKey**: 컴포넌트 조합 구성에 따른 데이터 조합 묶음을 분류 및 정의하고, std::bitset<256> 비트 마스킹을 활용해 타겟 아키타입을 O(1)에 조회 필터링.

[2. Archetype & ChunkData]

문제인식 및 필요성

- 단순 컴포넌트들을 타입별로 배열에 연속 적재할 경우, 엔티티별 데이터 조합 다양성으로 인해 메모리 정렬 및 수명 관리가 복잡해짐.
- CPU 공간 지역성(Spatial Locality) 성능을 보장하기 위해 물리 메모리 블록 크기를 제한하고 통제하는 관리 레이어 필요.

설계

- **16KB 물리 청크 단위 관리**: 동일한 컴포넌트 조합인 ArchetypeKey를 기반으로, 각 컴포넌트 데이터들을 **16KB(16384바이트)의 연속된 물리 메모리 블록(Chunk)** 구조로 분할 구성.
- **청크당 수용 한도 규격화**: 포함된 컴포넌트 중 가장 크기가 큰 타입을 기준으로 16KB 대비 역산하여 청크가 수용하는 최대 인덱싱 개수(m_ChunksCapacity)를 도출하고 모든 컴포넌트 청크의 인덱싱 단위 통일.
- **추상 인터페이스 기반 설계**: 다양한 타입의 청크 제어를 위해 ComponentChunk 추상 클래스를 수립하고 Swap, SwapAndPop 등의 공통 조작 가상 함수 정의.
- **용량 동적 확장**: 청크 용량(16KB) 초과 시, 추가 물리 청크 컨테이너를 생성하여 버퍼 용량을 점진적으로 자동 확장.

[3. 컴포넌트조합 및 엔티티 생성 흐름]

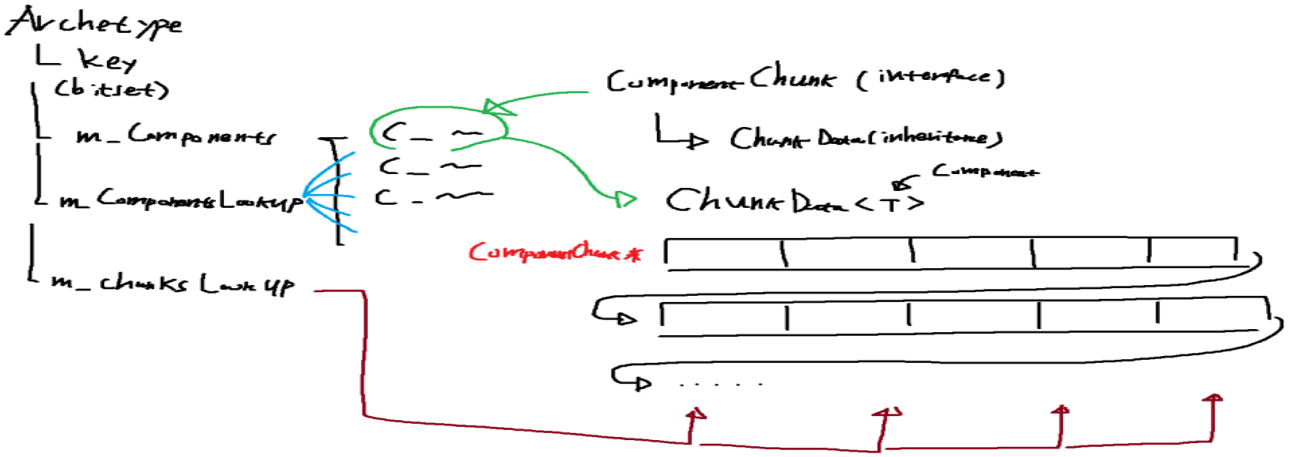
문제인식 및 필요성

- 다양한 컴포넌트 조합으로 엔티티를 생성하는 과정에서 가변 인자 템플릿 사용으로 인한 런타임 연산 복잡도 및 동적 할당 오버헤드 억제 필요.
- 복사/이동 오버헤드를 막기 위해 컴포넌트 등록 시 청크의 메모리 공간 사전 예약 및 참조 전달 보장 필요.

설계

- **컴파일 타임 폴드 표현식**: 가변 인자 템플릿과 C++17 폴드 표현식을 통해 컴포넌트 조합 식별 및 아키타입 등록 연산을 고속 최적화.
- **CreateArchetype**: 컴포넌트 가변 인자들을 전개하여 아키타입을 빌드하고 청크 슬롯들을 자동 매핑.

- **CreateEntity**: 룩업 테이블 상에 엔티티 식별자를 선할당하고, ReserveIndexes를 가동해 컴포넌트 청크의 물리 위치(Row, Col) 지정.
- **우측값 전달을 통한 AddComponent**: 컴포넌트 추가 등록 시, 청크 벡터의 마지막 인덱스 주소에 값을 직접 덮어씌우는 우측값 참조(&&) 전달을 통해 복사 및 불필요한 재할당 원천 차단.



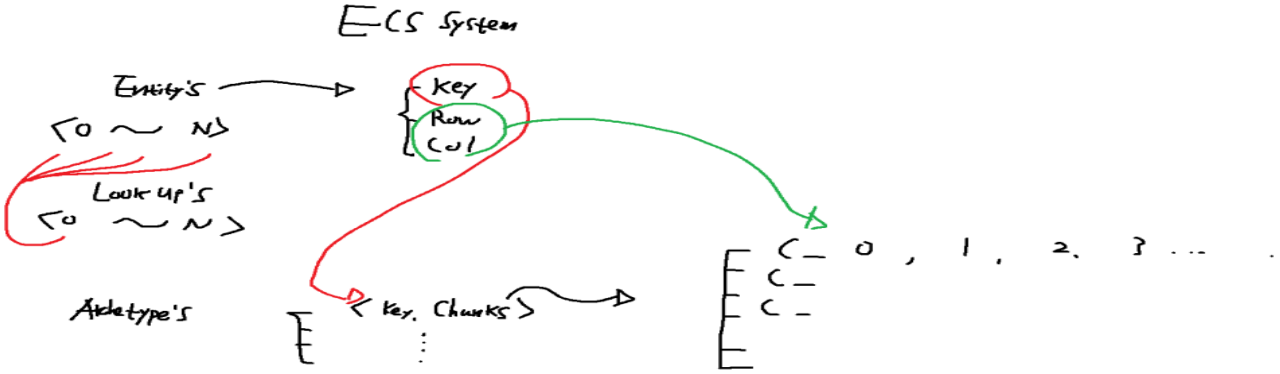
[4. Sparse/Dense(희소배열) 기반 구성]

문제인식 및 필요성

- 엔티티/컴포넌트의 빈번한 생성과 삭제 과정에서 중간 데이터 소멸 시 빈 슬롯을 채우기 위해 전체 메모리 블록을 시프트($O(N)$)하는 병목 오버헤드 발생.
- 데이터 밀집도를 유지하기 위한 시프트 과정에서 엔티티가 가리키는 실제 컴포넌트 청크 주소(Row/Col)가 어긋나는 정합성 왜곡 방지 필요.

설계

- **O(1) Swap & Pop 소멸 구조**: 빈 공간 시프트 제거를 위해 대상 데이터를 삭제하고 해당 청크 최후방(Back)의 유효 데이터를 그 자리에 복사해 채우는 구조 수립.
- **이중 참조 매핑 (Sparse & Dense)**:
    - **m_LookupTable(SparseArray)**: 엔티티 고유 ID를 인덱스로 활용해 DenseVector 내의 위치(EntityIdx)를 조회.
    - **m_Entitys(DenseVector)**: 실제 물리적 컴포넌트 청크의 위치 정보(행/열)를 타이트하게 압축해 순회 성능 확보.
- **Defragmentation (참조 정합성 보정)**:
    - 엔티티 삭제 시 룩업 인덱스를 해제하고 DeleteComponent 실행.
    - 최후방 청크 데이터를 삭제된 위치로 Swap & Pop으로 이동하고, 이동된 엔티티의 고유 ID를 룩업 추적 테이블을 거쳐 해당 Row/Col 값으로 즉시 동기화.



[5. 비트연산 기반 컴포넌트데이터조회(ArchetypeQuery)]

문제인식 및 필요성

- 특정 시스템(예: RenderSystem)이 요구하는 컴포넌트 조합을 순회할 때, 엔진 내의 전체 아키타입 풀을 선형 검색하는 방식은 치명적인 루프 오버헤드 유발.

설계

- **비트 AND 연산 쿼리**: 특정 컴포넌트 마스크(Key) 필터링에 비트 연산 (iter.first & key) == key를 적용하여 쿼리 검색 속도를 O(1) 수준으로 가속.
- **포인터 캐싱**: 매칭된 아키타입들의 물리 주소 포인터 컨테이너만 선별하여 해당 시스템의 쿼리에 즉시 전송함으로써 연속 메모리 다이렉트 접근 실현.

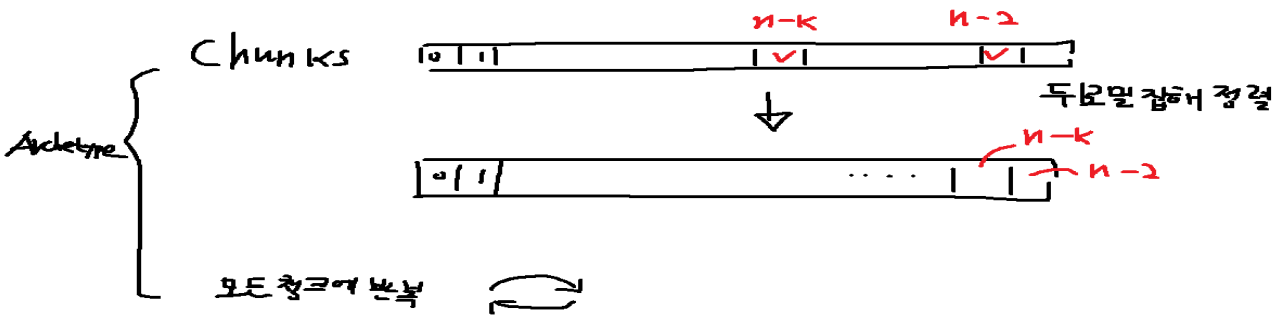
[6. 순회 최적화를 위한 청크 정렬 업데이트]

문제인식 및 필요성

- 매 프레임 모든 엔티티를 전수 순회할 필요가 없고, 특정 상태 변화(예: 해당 프레임에 움직임이 있는 활성 개체)가 감지된 일부 데이터만 선별 순회하는 효율적인 분할 루프 기법 요구.

설계

- **UpdateSwapChunk 구현**: 조건을 충족하는 엔티티들(Row/Col)을 청크 뒤편부터 순서대로 맞바꿔 연속 정렬하는 밀집도 최적화 구조 구축.
- **정합성 갱신**: 교환 과정에서 아키타입 내의 ChunkLookup과 ECSSystem의 룩업 테이블 간의 바뀐 물리 주소 인덱스 실시간 동기화 보장.
- **전파 인덱스 (transferIdx) 제어**: 시스템 순회 시 아키타입 내부의 전파 인덱스 변수를 경계값으로 사용하여, 전체 청크 중 밀집 정렬된 활성 데이터 세그먼트 구간만 한정 순회하도록 검색 오버헤드 최소화.



결과 및 개선여지

종합 활용 결과

- 16KB 단위 청크 기반의 선형 데이터 조밀화 배치를 통해 CPU L1/L2 캐시 히트를 극대화해 대량의 개체 처리 환경에서도 60fps 이상의 안정적인 성능 확보.
- 희소/밀집(Sparse/Dense) 룩업 테이블 및 Swap & Pop 기법 연동으로 개체 소멸 비용을 $O(1)$로 고정하고 참조 무결성 완벽 유지.
- ArchetypeKey 비트셋 쿼리로 수많은 이종 엔티티 조합 군에서 필요한 시스템 데이터군만 고속 추출하여 순차 순회 처리 지원.

개선여지

- **멀티스레드 시스템 병렬화**: 현재 단일 스레드 순차 처리 구조를 탈피하여, 각 시스템 간 컴포넌트 종속성(Dependency) 및 Read/Write 권한을 분석한 스레드 분배 멀티태스킹 설계 필요.
- **원시 포인터 및 커스텀 메모리풀 대체**: 데이터 청크의 STL 의존 디버깅용 안전장치를 걷어내고, 원시 버퍼 및 특화된 고성능 메모리풀 설계 적용을 통한 할당 비용 최적화 여지 존재.

개발과정 세부사항

DOD기반 아키텍쳐 설계 및 Archetype 아키텍쳐 설계 및 도입배경

ECS기반 시스템 및 세부구성

ECS시스템 코드스니펫 및 세부사항 오버뷰

03 [Engine/Render System및 RenderPass 구성]

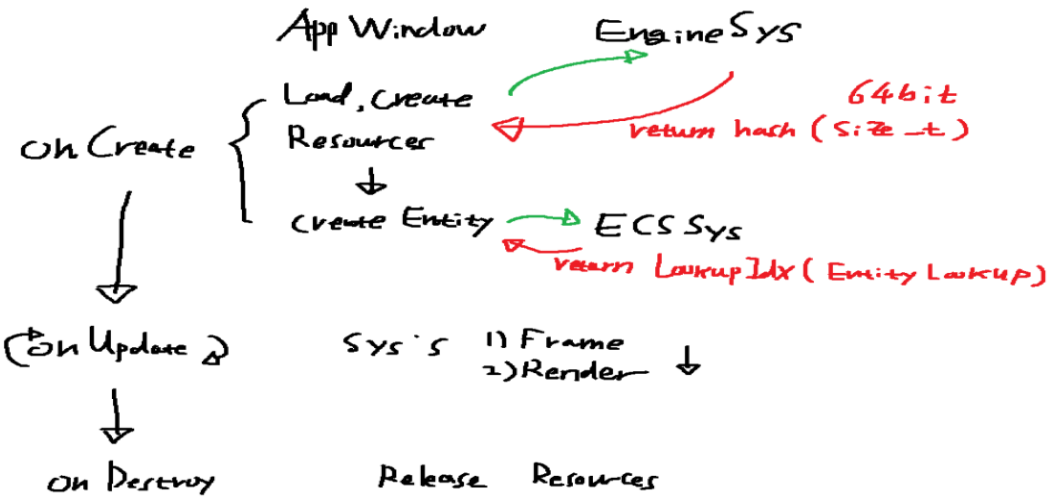
[1. EngineSystem 구성]

문제인식 및 필요성

- **객체 단위 자원 소유의 한계**: 개별 객체가 자원을 독립 소유 시 중복 로드 여부 판별이 불가능하며, 자원 수명 관리 및 디버깅 오버헤드 급증.
- **책임 분리 필요성**: 프로젝트 규모 확장에 따라 자원 관리 주체(EngineSystem)와 자원 사용 주체(RenderSystem)를 분리하여 설계 일원화 필요.

설계

- **중앙 관리 매니저 패턴**: FNV-1a 해싱을 통한 캐싱 구조로 단일 인스턴스 보장 및 중복 생성 방지.
- **64비트 정수형 해시 키**: 디스크 상의 파일 경로 및 자원 명칭을 FNV-1a 64비트 해시로 변환하여 에셋 고유 식별자 구축.
- **중복 I/O 차단**: 로드된 모든 가상 자원 및 D3D11 래퍼 객체를 내부 해시맵(Map) 컨테이너에 캐싱하고, 해시 키 기반 O(1) 쿼리로 파일 중복 적재 원천 제어.



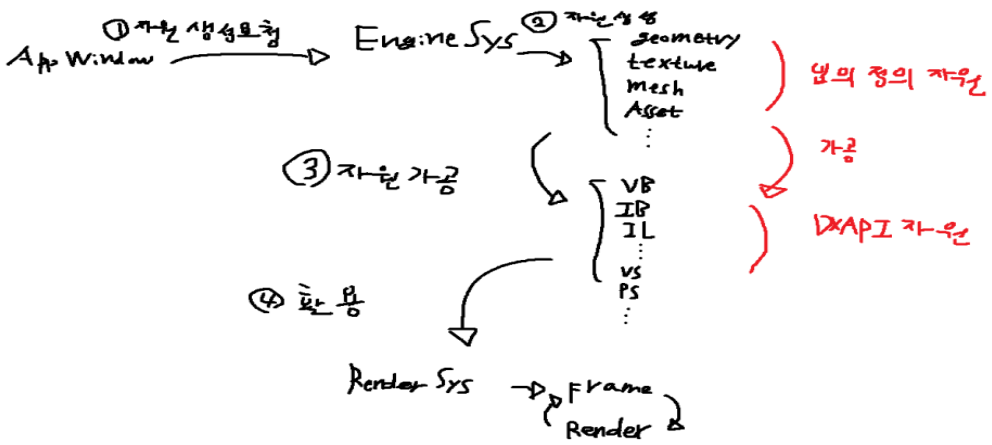
[2. 리소스 가공 및 파이프라인에서의 자원 조회/바인딩]

문제인식 및 필요성

- ResourceSystem에서 로드한 가상 정의 에셋(원시 데이터)은 GPU 파이프라인에 직접 바인딩 불가능하므로, D3D11 물리 자원으로의 가공 레이어 필요.
- 타 서브시스템 간 자원 의존성(Coupling)을 소거하고 자원 동기화 무결성 보장 필요.

설계

- **D3D11 래퍼 클래스 관리**: 원시 에셋을 기반으로 D3D11 물리 객체(버퍼, 뷰, 렌더 상태 등)를 생성하고 소유권을 EngineSystem으로 고정.
- **자원 참조 채널 일원화**: RenderSystem 및 CollisionSystem 등 자원 사용 객체는 실물 데이터를 소유하지 않고, 오직 고유 해시 키 및 간접 접근 함수를 통해서만 객체를 참조 및 바인딩하도록 설계.
- **공용 자원 사전 선점**: 상수 버퍼, 기본 뷰 포트 등 시스템 전반에서 공용되는 비귀속 자원들을 초기화 단계에서 선할당 및 해시 맵 등록 처리하여 정합성 유지.



[3. RenderSystem 구성 및 렌더패스(Sort By Key) 설계]

문제인식 및 필요성

- OOP에서 ECS로 개편 직후, 아키타입 청크 전체를 나이브하게 쿼리하여 바인딩하는 방식으로 인해 D3D11 상태(State) 교체 비용 병목 및 중복 바인딩 오버헤드 미해결.
- 드로우콜 오버헤드를 극소화하기 위해 렌더링 과정을 패스(Pass) 단위로 쪼개고 정렬하여 일괄 처리하는 파이프라인 재설계 필요.

설계

- **렌더패스(RenderPass) 정립**: 현대 상용 엔진 스타일의 패스 분류(Shadow, Forward, Deferred, Cubemap, Outline 등)에 맞춘 렌더 파이프라인 단계 수립 및 드로우콜 단위인 RenderItem 구조 도입.
- **64비트 정렬 키 설계**: 자원의 상태 전환 중요도와 D3D11 파이프라인 상태 변경 비용 가중치에 따라 비트 영역을 시프트하여 64비트 정수 정렬 키 생성.
- **정렬 키 비트셋 구조**:
  - **Pass (4bit)**: 최상위 비트. 렌더 타겟 및 뷰포트 변경 비용 최적화를 위한 패스별 분기.
  - **Shaders (16bit)**: 머티리얼에 결합된 VS, HS, DS, GS, PS 파이프라인 조합 해시 ID.
  - **States (8bit)**: Rasterizer, Blend, DepthStencil 렌더 상태 조합 ID.
  - **Resources (16bit)**: 결합될 Mesh(VB/IB), ConstantBuffer 및 Texture 리소스 해시 ID.
  - **Distance (20bit)**: 카메라와 오브젝트 간의 선형 깊이 거리 값 (투명 패스는 Back-to-Front, 불투명 패스는 Front-to-Back 정렬 및 early-Z 극대화).
- **서브 세그먼트 키 발급**: 가상자원 해시값(64비트)을 정렬 키에 그대로 대입하지 않고, 고유 식별을 위해 GetRenderPassKey_Shaders/States/Resources 해시 테이블 룩업 기반의 컴팩트한 서브 ID 실시간 발급.

[4. RenderQueue 기반의 드로우콜 구성]

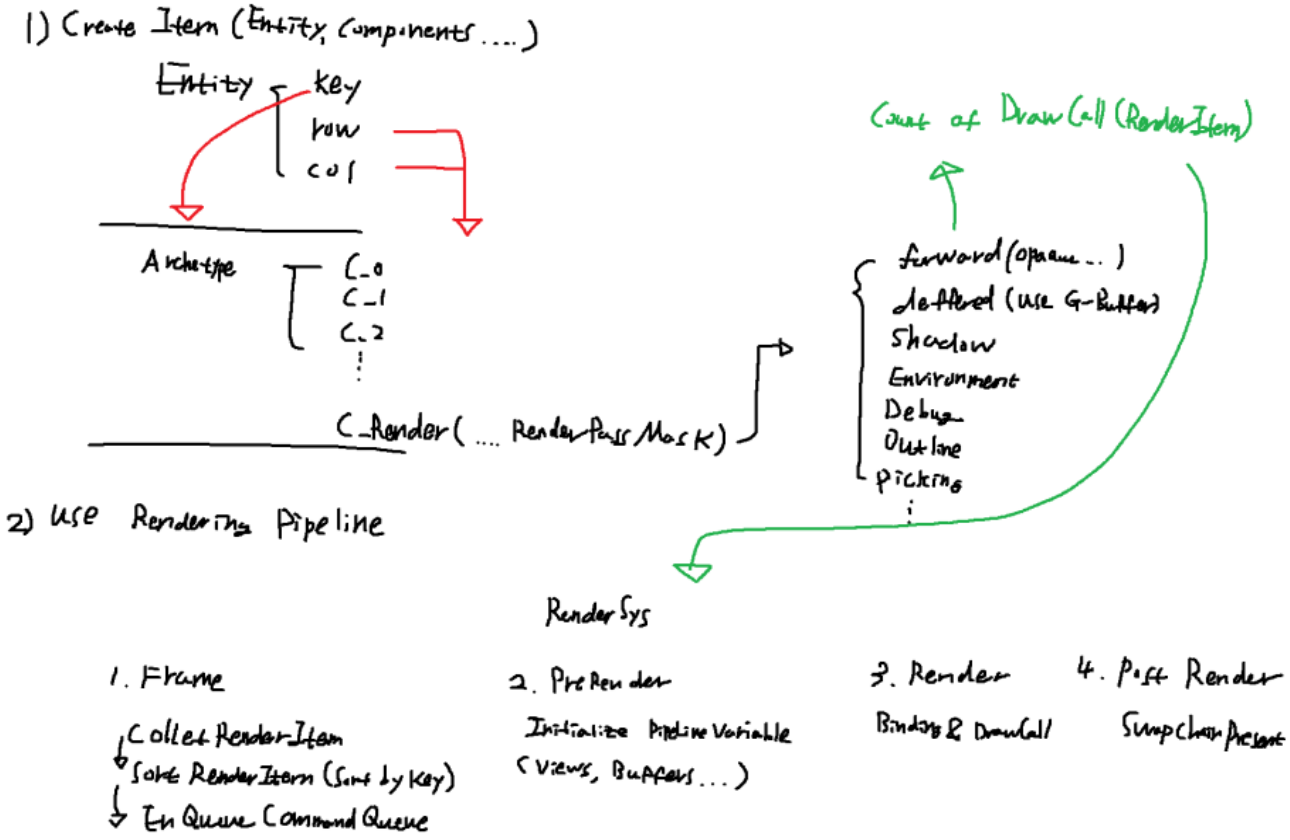
문제인식 및 필요성

- 생성된 64비트 정렬 키를 기반으로 리소스와 상태가 동일한 드로우콜들을 병합(배칭)하여 하드웨어 호출 오버헤드 차단 필요.
- 매 프레임 발생하는 불필요한 행렬 연산 및 반복적인 바인딩 통제 필요.

설계

- **렌더 아이템 수집 (CollectRenderItem)**:
  - _ECSSystem.QueryArchetypes를 통해 렌더링 대상 컴포넌트(C_Transform, C_Render) 보유 아키타입군 고속 검색.
  - 엔티티의 렌더 가능 상태(bRenderable) 및 렌더 패스 마스크(passMasks) 분기를 판별하여 각 패스별 정렬 키 조립 후 임시 버퍼 적재.

- **인스턴싱 배치 큐 전개 (EnqueueRenderItem)**:
    - 정렬 키의 상위 자원 비트가 동일한 개체들을 그룹화하여 드로우 개수(instanceCnt)와 버퍼 내 오프셋(batchIdx)을 산출해 명령 큐(m_hRP_CommandQueue) 빌드.
    - 배칭 대상 인스턴스들의 월드 행렬을 구조화 버퍼(stb_worldmats)에 일괄 복사(memcpy)하고 단 1회의 바인딩으로 일괄 드로우(DrawIndexedInstanced) 기동.



[5. 명령 큐 기반 렌더링 및 추가 최적화]

문제인식 및 필요성

- 프레임당 수천 번의 루프를 돌며 동일 상태 값을 파이프라인에 중복 설정하는 불필요한 API 호출 낭비 근절 필요.

설계

- **XOR 비트 연산 기반 지연 바인딩 (Lazy Binding)**:
    - 렌더 큐 순회 루프 내부에서 직전 정렬 키(keyPrev)와 현재 정렬 키(keyCur) 간의 비트 XOR 연산(keyCur ^ keyPrev) 수행.
    - 비트 변화 오프셋(60, 44, 36, 20)을 검출해 차이가 발생한 파이프라인 단계(Pass, Shader, State, Resource)만 분기하여 바인딩 API 선별 호출.
- **캐싱 포인터 얼리 리턴 (Early Return)**:
    - D3D11 래퍼 객체 내부에 직전 바인딩된 실제 물리 주소를 캐싱용 상태 변수로 유지.
    - 바인딩 요청 발생 시, 해당 변수와 목적지 주소 값을 비교해 동일할 경우 API 호출 없이 조기 리턴하여 드라이버 부하 최소화.

결과 및 개선여지

종합 활용 결과

- 파일 경로 FNV-1a 해싱 기반 자원 일원화 관리로 메모리 중복 적재를 완전히 제거하고 누수 안정성 및 수명 정합성 확보.
- 64비트 정렬 키 기반 렌더패스(Sort-by-Key) 도입을 통해 파이프라인 상태 전환 효율성을 확보하고 추가 렌더패

스 확장에 유연한 아키텍처 수립.
- XOR 비트 연산 차분 바인딩 및 캐싱 포인터 비교 제어로 D3D11 API 바인딩 호출 오버헤드를 극소화.
- 동일 리소스 대상 인스턴싱 드로우콜 배칭을 통해 드로우콜 횟수를 감소시켜 렌더링 병목 해소.

개선여지

- **더티 플래그(Dirty Flag) 행렬 캐싱**: 매 프레임 인스턴스별 월드 행렬을 전수 계산하는 연산 낭비를 줄이기 위해, 데이터 변경 발생 시에만 행렬을 갱신하는 캐싱 구조 도입 필요.
- **지연 컨텍스트(DeferredContext) 가동**: 렌더 명령 생성(Command List)의 CPU 병목을 분산하기 위해 즉각 컨텍스트(ImmediateContext) 의존도를 낮추고 멀티스레드 지연 컨텍스트 렌더링 도입을 통한 파이프라인 고도화

개발과정 세부사항

커맨드큐 기반 렌더패스 설계
드로우콜 인스턴싱 배치 및 최적화
EngineSys세부 구성 및 오버뷰

04 [가시성/충돌 판정 최적화 및 그래픽스 파이프라인]

[1. 가시성/경계볼륨 판정]

문제인식 및 필요성

- **불필요한 GPU 바인딩 제거**: 카메라 시야 밖에 존재하는 기하 자원까지 매 프레임 GPU 파이프라인에 바인딩하는 행위는 CPU/GPU 양측에 무의미한 연산 부하 유발.
- **피킹 연산 효율화**: 월드 공간 내 특정 객체 선택 시, 모든 객체의 메시 정점을 전수 조사하는 방식의 성능 병목 제어 필요.

설계

- **단계별 충돌 판정 구조**: 절두체 컬링 -> 경계 볼륨 검사 -> 정밀 충돌 검출 순서로 부하를 단계적 필터링.
- **선형 깊이 기반 절두체 평면 유도**: 원근 투영 변환의 비선형 깊이 분포를 선형 보정한 투영 행렬을 빌드하고, 뷰-투영 행렬의 전치행렬(Transpose)을 활용해 6개 절두체 평면 방정식을 수학적으로 도출.
- **경계 볼륨(Bounding Volume) 테스트**: 로컬 공간의 AABB(Box) 또는 Sphere 수치를 월드 변환 행렬과 승산하여 절두체 평면 교차 여부를 판별하여 1차 가시성 통제.
- **단계적 오브젝트 피킹 (Picking)**:
    - 화면 좌표계 입력 값 역산: Viewport -> NDC -> Proj -> View 공간으로의 역변환을 수행해 월드/로컬 공간의 피킹 레이(Ray) 공식 유도.
    - 1차 검사: Ray-Sphere(내적 및 피타고라스 기하식) 및 Ray-AABB(Slab Method) 교차 테스트 수행.
    - 2차 검사: 1차 검사를 통과한 액터만 대상으로 Ray-Triangle(Möller-Trumbore 알고리즘) 정밀 교차 검출 기동.

[2. 계산 셰이더 기반 충돌 판별]

문제인식 및 필요성

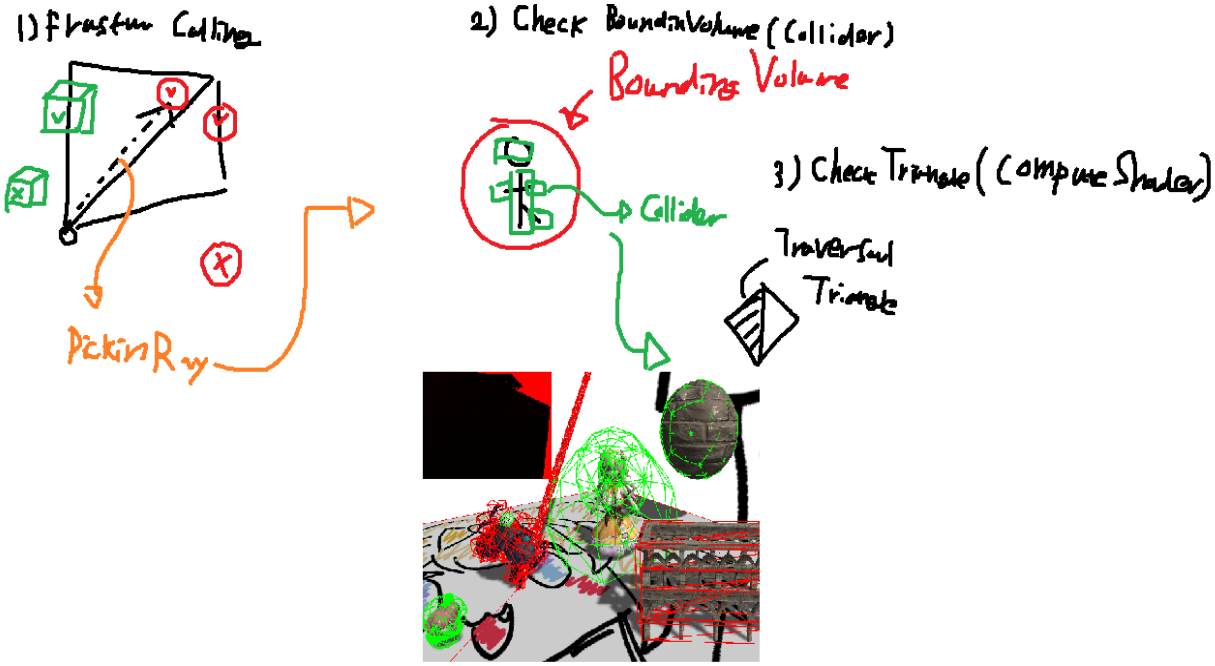
- **CPU 연산 병목**: 경계 볼륨 통과 후 다량의 스키닝 정점 계산과 수천~수만 개의 삼각형 교차 검출을 단일 스레드 CPU 연산으로 처리 시 프레임 드랍 발생.
- **병렬 가속**: 정점 변형(Skinning) 및 삼각형 피킹 연산을 GPU 다중 스레드 연산으로 이관하여 가속화 필요.

설계

- **GPU 스키닝 및 교차 비동기 병렬화**: D3D11 Compute Shader(CS) 파이프라인 구축.
- **정점 및 뼈대(Bone) 버퍼 구조화**: 메쉬 정점 및 애니메이션 본 행렬 데이터를 구조화 버퍼(StructuredBuffer) 형태로 GPU 메모리 업로드.
- **스레드 분배 및 연산**:
    - Dispatch 호출 시 총 삼각형 개수와 GPU 스레드 그룹 크기를 매칭하여 64개 스레드 단위로 연산 분배.
    - 계산 셰이더 내에서 4가중치 본 스키닝 위치 변형(ComputeSkinning)을 수행하고, 꼭지점 3개와 피킹

레이 간의 Möller-Trumbore 알고리즘을 계산해 충돌 여부 및 최소 깊이(fDist) 검출.

- **StagingBuffer기반 CPU readback :** 검출 결과를 UAV 버퍼에 기록한 뒤, D3D11 CopyResource를 사용해 Staging 버퍼로 복사하고 Map 함수로 CPU 영역에 수집 및 교차 좌표 검출.



[3. 렌더 패스 세부구현]

문제인식 및 필요성

- **포워드 렌더링 한계**: 다중 광원 연산 시 픽셀 셰이더 재실행 부하 및 불필요한 D3D11 셰이더/상태 전환 오버헤드 완화 필요.

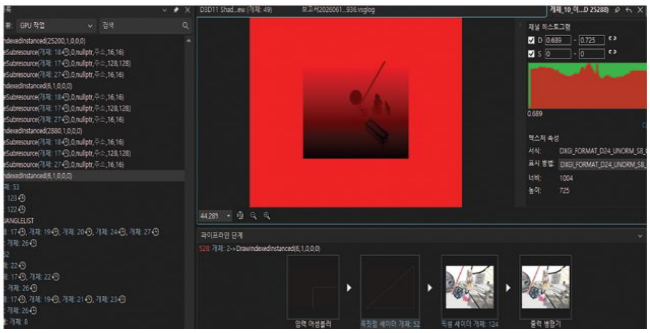
설계

- **64비트 정렬 키 기반 5대 핵심 렌더패스 라우팅**:
  - **Deferred Rendering (Opaque)**: 4개 채널 G-Buffer(Position, Normal, Albedo, Specular)에 동시 MRT 출력 후, 별도의 스크린 쿼드 조명 패스(DeferredLighting)에서 픽셀 단위 셰이딩 처리.
  - **Shadow**: RTV 바인딩 해제(nullptr 지정) 상태로 광원 시점 깊이 맵만 빌드하여 3x3 PCF(Percentage Closer Filtering) 픽셀 샘플링 적용.
  - **Environment Mapping (CubeMap)**: 기하 셰이더(GS)의 SV_RenderTargetArrayIndex를 활용하여 단일 기하 패스를 6개 RTV 면으로 병렬 렌더링하고 동적 밉맵 생성.
  - **Outline**: 2패스 기법 적용. 1패스(Outline_Write)에서 스텐실 마킹 후, 2패스(Outline_Draw)에서 정점 셰이더의 법선 방향 팽창(extrudedPos)을 사용해 실루엣 외곽선 단색 드로우.
  - **Debug**: 디버그 메쉬(Box/Sphere) 출력을 위해 별도의 정점/인덱스 버퍼를 생성하지 않고, 기하 셰이더(Box) 및 도메인 셰이더 테셀레이션(Sphere 구형 보간 투영)을 이용해 버퍼리스 렌더링 처리.
  - **Forward Rendering (Sky, Transparent, UI)**: RTT(Render To Texture) 기법으로 스크린 쿼드 사양의 백 버퍼 포워딩 및 블렌딩 처리.

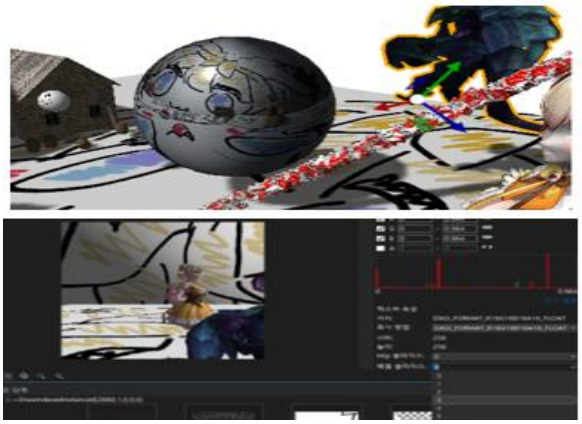
1. Deferred Shading & G-Buffer 파이프라인 연동



2. Shadow Pass와 투영 텍스처링 PCF 기법



3. Environment Map(CubeMap) Pass



4.Outline Pass



결과 및 개선여지

종합 활용 결과

- 선형 깊이 절두체 평면 설계와 단계별 경계 볼륨 가시성 판정 도입으로 불필요한 드로우콜을 사전에 차단하여 최적화 실현.
- 좌표계 역산 피킹 레이 설계로 정확한 마우스 3D 공간 충돌 매핑 검증.
- CPU 연산 한계를 초과하는 스키닝 및 폴리곤 피킹 검출을 GPU Compute Shader 병렬 연산으로 이관하여 연산 정밀도와 프레임 레이트 방어 성공.
- MRT 디퍼드, PCF 그림자, 6면 환경 매핑, 스텐실 카툰 아웃라인, 테셀레이션 디버그 렌더링을 렌더패스 규격 하에 유기적으로 구동하여 비주얼 완성도 입증.

개선여지

- **Staging Buffer 동기화 대기(Stall) 소거**: 현재 Compute Shader 완료 후 Map 호출 시 발생하는 CPU-GPU 동기화 병목을 제거하기 위해, 동기화 펜스(Fence) 및 비동기 쿼리를 적용한 수 프레임 지연 읽기(Latency Tolerant Readback) 아키텍처로 개선 필요.
- **공간 분할 구조 도입**: 월드 공간 내 모든 오브젝트 대상 $O(N)$ 전수 조사를 방지하기 위해 옥트리(Octree) 또는 BSP 공간 분할 트리 구조를 도입하여 충돌 탐색 오버헤드 최소화 여지 존재.

개발과정 세부사항

FBX임포트 및 스키닝애니메이션 구조

그림자매핑(PCF샘플러)

환경맵(큐브맵)

조명연산(Directional, Point, Specular)

노말매핑

Gbuffer기반 디퍼드렌더링, 포트스프로세싱, 조명연산

충돌체 및 오프젝트피킹

GPU(계산셰이더)기반 충돌판정 및 아웃라인/피킹 트라이앵글

패스별 세부설명 및 오버뷰(작업정리6)